

ZAP! Advanced Programming Topics

The author of this software stands in solidarity with  Ukraine. We believe in a world where international borders are respected and human rights are upheld. We encourage all users of this software to contribute to humanitarian efforts in  Ukraine.

Advanced Concepts and Techniques for Expert ZAP! Programmers

Version: 1.0

Date: January 2026

Table of Contents

1. [Pointer Operations](#)
 2. [Memory Management](#)
 3. [Inline Assembly](#)
 4. [Hardware Access](#)
 5. [Optimization Techniques](#)
 6. [Module System Deep Dive](#)
 7. [Performance Profiling](#)
 8. [Advanced Patterns](#)
 9. [Debugging Assembly Output](#)
 10. [Compiler Internals](#)
 11. [Memory Allocation Details](#)
 12. [Safety Features](#)
 13. [Debugger Symbol Support](#)
-

Pointer Operations

Understanding Pointers

Pointers store memory addresses. In ZAP!, pointers are 16-bit values (word-sized):

```
byte x = 42
byte ^ptr          ; Pointer type
ptr = @x           ; Store address of x (address-of operator)
byte value = ptr^   ; Dereference (read from address)
ptr^ = 99          ; Dereference (write to address)
```

Taking Addresses

The `@` operator (address-of) retrieves an address:

```
byte data = 100
byte ^ptr = @data    ; Get address of data, store in ptr
```

What's stored in **ptr**?

- The 16-bit memory address where **data** lives
- Compiler assigns this address automatically

Note: The **@** operator is the only supported syntax for taking addresses. Use **@var** (not **^var**) in expressions and initializers.

Getting Addresses of Complex Expressions

The **@** operator works with arrays and struct fields:

```
; Array element address
byte arr[] = {10, 20, 30}
word elem_addr = @arr[1]    ; Address of arr[1]

; Struct field address
struct Point
    byte x
    byte y
end

Point p = {50, 100}
word field_addr = @p.x      ; Address of p.x field
```

Dereferencing

The **^** operator also reads/writes through pointers:

```
byte x = 50
byte ^ptr = @x

; Read through pointer
byte value = ptr^    ; value = 50

; Write through pointer
ptr^ = 99
; Now x = 99
```

Pointer Arithmetic

Pointers support strict C-style addition and subtraction. **Type matters:**

```

; BYTE pointer: +1 moves 1 byte
byte arr[] = {10, 20, 30}
byte ^ptr = @arr
ptr = ptr + 1          ; Points to second element
byte second = ptr^     ; second = 20

; WORD pointer: +1 moves 2 bytes (word size)
word addresses[] = {$1000, $2000, $3000}
word ^wptr = @addresses
wptr = wptr + 1        ; Points to second WORD (2 bytes later)
word addr2 = wptr^     ; addr2 = $2000

; Pointer Difference: Distance in elements
word distance = wptr - @addresses ; Returns 1 (1 element apart)

```

The scaling is automatic:

- `byte ^ptr + 1` moves 1 byte
- `word ^ptr + 1` moves 2 bytes
- `long ^ptr + 1` moves 4 bytes
- `struct Point ^ptr + 1` moves by the `sizeof(Point)`

Pointer comparisons: Pointers cleanly support all relational operators (`==`, `!=`, `<`, `>`, `<=`, `>=`) against other pointers or the literal constant `0`. Comparing a pointer to an arbitrary numeric threshold or scalar variable is heavily restricted by the semantic checker.

Pointer Arrays

Array of pointers to data:

```

byte data1[] = "First"
byte data2[] = "Second"
byte ^ptrs[2]
ptrs[0] = @data1
ptrs[1] = @data2

; Access through array
byte char1 = ptrs[0]^ ; 'F' from data1
byte char2 = ptrs[1]^ ; 'S' from data2

```

Note: Pointer arrays are stored in zero page if they fit; otherwise they are placed in BSS. ZP-only optimizations apply only when the array is in zero page.

Pointer to Pointer (Limited)

ZAP! doesn't support full pointer-to-pointer chains, but you can simulate:

```
byte x = 100
byte ^ptr1 = @x
byte ^ptr2 = ptr1      ; NOT pointer to ptr1, but copy of address
```

Common Pointer Pitfalls

Invalid Dereference:

```
byte x = 10
byte y = ^x      ; ERROR: y is byte, not pointer!
```

Non-ZP Pointers:

```
byte ^ptr @$2000    ; Fixed address, not in zero-page
byte v = ptr^       ; Works via TMP0 (slower), no ZP-only optimizations
```

Pointer Optimization

The compiler can optimize pointer usage:

```
; Compiler optimizes this:
byte ^ptr = @array
byte val1 = ptr^
ptr = ptr + 1
byte val2 = ptr^

; Into:
; LDA array+0      ; Load first value
; LDA array+1      ; Load second value
```

Advanced Example: Linked List

```
; Simple linked list node structure (simulated with arrays)
byte value[10]           ; Node values
byte next_index[10]      ; Next node index (255 = end)
byte list_start = 0      ; Start of list

proc traverse_list()
    byte current = list_start

    while current != 255
        byte node_value = value[current]
        current = next_index[current]
```

```
end
end
```

Memory Management

Zero-Page (ZP)

Special 256-byte area for fast operations:

```
$00-$FF = Zero-page
```

Fastest and smallest code for:

- Variables
- Pointers
- Temporary values

```
byte x           ; Goes to zero-page (if space available)
byte arr[256]    ; Too big - goes to BSS (high RAM)
```

ZP Allocation Order

Compiler allocates in this order:

1. **Pointers** - ZP if they fit; otherwise BSS
2. **Byte variables** - ZP first, then BSS
3. **Word variables** - ZP first, then BSS
4. **Arrays** - Always BSS (pointer arrays go to ZP only if they fit)
5. **Temporaries** - TMP0-TMP4 in ZP

Example layout:

```
$00-$01    System variables
$02-$03    User pointer 1
$04-$05    User pointer 2
$06        User byte variable
$07-$08    System temporaries (if used)
$09+       More variables
...
$FF        Last ZP byte

High RAM   Arrays and strings
```

BSS (Uninitialized RAM)

Memory pool starting at high address (e.g., \$3000):

```
byte large_array[256] @0x3000 ; Fixed address
byte another[512] ; Auto-placed in BSS
```

Fixed-Address Variables

Explicitly place variables for hardware access:

```
byte GTIA_HPOS0 @$D000 ; Hardware register
byte screen_data[256] @$4000 ; Screen memory

proc set_player_pos(byte x)
    GTIA_HPOS0 = x ; Direct hardware access
end
```

Benefits:

- Access hardware-mapped registers
- Reserve specific memory for cartridge/OS
- Create custom memory maps

Memory Collision

Be careful with fixed addresses:

```
byte var1 @$2000
byte var2 @$2000 ; ERROR: Collision!
```

Dynamic Memory (Not Supported)

ZAP! doesn't support malloc/free. All memory is static.

Static Local Variables (Persistent State)

Static variables are local variables that retain their values between procedure calls. Unlike regular local variables (which are re-initialized on each entry), static variables are initialized only once at program startup.

Syntax and Behavior:

```
proc increment_counter()
    static byte counter = 0 ; Initialized once at program start
    counter = counter + 1
    return counter
end
```

```

proc main()
    byte x1 = increment_counter() ; Returns 1, counter now = 1
    byte x2 = increment_counter() ; Returns 2, counter now = 2
    byte x3 = increment_counter() ; Returns 3, counter now = 3
end

```

Initialization Flow:

1. Program starts
2. Global variables are initialized
3. Static local variables are initialized (in order of procedures)
4. CONSTRUCTORS are called in the same order as as .include includes modules
5. MAIN is called
6. After MAIN ends, endless loop is performed

Rules:

- Only for **local variables** (inside procedures/functions)
- Cannot combine with **CONST** modifier
- Must have an initializer (static variables can't be uninitialized)
- Storage is in zero-page (ZP) if space available, otherwise in BSS

Use Cases:

1. Call counters:

```

proc get_object_id()
    static byte next_id = 1
    byte id = next_id
    next_id = next_id + 1
    return id
end

```

2. State machines:

```

proc game_state()
    static byte state = 0 ; 0=menu, 1=playing, 2=paused

    if state == 0
        ; Handle menu
        state = 1
    elseif state == 1
        ; Handle game
        if player_pressed_pause
            state = 2
        end
    elseif state == 2
        ; Handle pause
    end
end

```

```

        if player_pressed_resume
            state = 1
        end
    end
end
end

```

See also ENUM and SWITCH for state machines implementation.

3. Resource pools:

```

const byte MAX_SPRITES = 10

proc allocate_sprite()
    static byte sprite_count = 0
    byte id

    if sprite_count < MAX_SPRITES
        id = sprite_count
        sprite_count = sprite_count + 1
        return id
    else
        return 255 ; Error code
    end
end
end

```

Memory Considerations:

Static local variables occupy memory like global variables - they're allocated in zero-page or BSS at compile time, not on the stack. Therefore:

- Each static variable takes permanent memory
- Static variables are always available (no stack overhead)
- Good for values that need to persist but shouldn't be global

Inline Assembly

Basic ASM Block

Embed raw 6502 assembly:

```

proc setup_display()
    asm
        LDA #$40
        STA $D400
    end
end

```

Assembly Labels

Create local labels:

```
proc loop_example()
    byte i = 0

    asm
    LOOP:
        INC $D000
        DEC _LOOP_EXAMPLE_I      ; Decrement local i (internal name:
        _LOOP_EXAMPLE_I)
        BNE LOOP
    end
end
```

Accessing ZAP Variables

Variables are prefixed with `_`:

```
byte my_var = 0
word my_word = 0

asm
    LDA _MY_VAR      ; Load byte variable
    LDX _MY_WORD     ; Load low byte of word
    LDY _MY_WORD+1   ; Load high byte of word
end
```

Assembly Label Naming Convention

The ZAP compiler uses a systematic naming convention to prevent collisions between your source code identifiers and compiler-generated labels. Understanding this convention is essential when writing inline assembly and referencing symbols.

The Rules

1. **Global identifiers** (global variables, procedures, functions) are prefixed with a **single underscore** (`_`), e.g. `my_var` → `_MY_VAR`
2. **Local identifiers** (local variables, parameters) use a **dollar sign** (`$`) to separate the procedure name from the variable name, e.g. local `result` in proc `compute` → `_COMPUTE$RESULT`
3. **Compiler-Generated Identifiers** (internal labels, temps, runtime helpers) are prefixed with **double underscore** (`__`)

The `$` separator for locals prevents naming collisions that would occur with underscore-only naming (e.g. global `test_result` and local `result` in proc `test` would both become `_TEST_RESULT` with underscores,

but with `$` the local becomes `_TEST$RESULT`). This requires `.FEATURE dollar_in_identifiers` in ca65, which the compiler emits automatically.

Source Identifiers: Single Underscore Prefix

All symbols declared in your ZAP code receive a single `_` prefix in assembly:

```
byte counter = 0
word address = $2000
byte data[] = {1, 2, 3}

proc initialize()
    counter = 0
end

func byte calculate()
    return counter + 10
end

proc main()
    asm
        ; Access variables with _ prefix
        LDA _COUNTER
        STA _ADDRESS

        ; Call procedures/functions with _ prefix
        JSR _INITIALIZE
        JSR _CALCULATE

        ; Access array with _ prefix
        LDA _DATA
        LDX _DATA+1
    end
end
```

Key Points:

- Global variables: `my_var` → `_MY_VAR`
- Local variables: `result` in `compute()` → `_COMPUTE$RESULT`
- Parameters: `index` in `use_lookup()` → `_USE_LOOKUP$INDEX`
- Procedures: `setup()` → `_SETUP`
- Functions: `get_value()` → `_GET_VALUE`

Compiler-Generated Identifiers: Double Underscore Prefix

The compiler generates many internal labels and helpers that use `__` prefix. You generally **should not reference** these directly, but knowing them helps avoid conflicts and assists debugging.

System Temporaries

Core 6502 working registers in zero page:

```
__TMP0, __TMP1, __TMP2, __TMP3, __TMP4, __TMP5 ; Multi-byte temporaries
__MATH_STACK ; Math expression evaluation stack (8 bytes)
__MATH0 ; Math operand 0 (4 bytes)
__MATH1 ; Math operand 1 (2 bytes)
```

Example (advanced usage only):

```
asm
    ; Compiler uses these internally - avoid in most cases
    LDA __TMP0
    STA __TMP1
end
```

Runtime Helper Routines

The compiler generates helper routines for complex operations:

Arithmetic (16-bit):

```
__ADD16 ; 16-bit addition
__SUB16 ; 16-bit subtraction
__MUL8 ; 8-bit multiplication
__MUL16 ; 16-bit multiplication
__DIV8 ; 8-bit division
__DIV16 ; 16-bit division
__MOD8 ; 8-bit modulo
__MOD16 ; 16-bit modulo
```

Array/Memory Operations:

```
__COPY_BYTES ; Block memory copy (TMP0=src, TMP2=dst, X=count)
```

Comparison Helpers:

```
__CMP16_EQ ; 16-bit equality
__CMP16_LT ; 16-bit less-than
__CMP16_GT ; 16-bit greater-than
```

Control Flow Labels

Control structures generate labels with `__ZAP_` prefix:

```
proc example()
  byte i = 0
  while i < 10
    i = i + 1
  end

  if i == 10
    ; Do something
  end
end
```

Generates assembly labels like:

```
__ZAP_while_1:
__ZAP_while_end_1:
__ZAP_if_2:
__ZAP_then_2:
__ZAP_else_2:
__ZAP_if_end_2:
```

Other control flow prefixes:

- `__ZAP_for_*` - FOR loops
- `__ZAP_switch_*` - SWITCH statements
- `__ZAP_case_*` - CASE labels
- `__ZAP_REL_TRUE_*` - Boolean short-circuit evaluation

Loop Temporary Variables

FOR loops create temporary variables:

```
proc count()
  byte i
  for i = 0 to 10
    ; Loop body
  end
end
```

Generates:

```
__COUNT_FOR_END_1    ; End value
__COUNT_FOR_STEP_1   ; Step value
```

String and Array Data Labels

Literal strings and array initializers in ROM:

```
byte msg[] = "Hello"
byte data[] = {1, 2, 3, 4, 5}
```

Generates:

```
__STR_DATA_1:    .byte "Hello", 0
__ARRAY_DATA_2:  .byte 1, 2, 3, 4, 5
```

Shared Local Variable Slots

When the compiler optimizes locals by sharing memory slots:

```
__LVSLLOT_1:    .res 2
__LVSLLOT_2:    .res 1
```

Practical Examples

Calling a procedure from assembly:

```
proc clear_screen()
    ; Clear screen code
end

proc main()
    asm
        JSR _CLEAR_SCREEN    ; Note: single _ prefix
    end
end
```

Accessing local variables (note \$ separator):

```
proc compute()
    byte result = 0
    word total = 0

    asm
        LDA #42
        STA _COMPUTE$RESULT    ; Local variable: _PROC$VAR
```

```

        LDA #$00
        STA _COMPUTE$TOTAL
        LDA #$10
        STA _COMPUTE$TOTAL+1
    end
end

```

Passing through a compiler temp (advanced):

```

proc unsafe_temp_access()
    asm
        ; Store something in compiler temp
        LDA #$FF
        STA __TMP0

        ; WARNING: Compiler may overwrite __TMP0 after ASM block!
    end

    byte x = some_pointer^ ; This operation may use __TMP0
end

```

Referencing array data:

```

byte lookup[] = {10, 20, 30, 40, 50}

proc use_lookup(byte index)
    asm
        LDX _USE_LOOKUP$INDEX    ; Parameter: _PROC$PARAM
        LDA _LOOKUP,X            ; Global array: _NAME
    end
end

```

Best Practices

1. **Always use `_` prefix** when referencing your ZAP variables, procedures, and functions
2. **Avoid referencing `__` prefixed symbols** unless you're doing advanced optimization
3. **Never create labels** in ASM blocks that start with `_` or `__` to avoid collisions
4. **Use descriptive local labels** in ASM blocks (e.g., `LOOP`, `SKIP`, `DONE`)
5. **Be aware of temp usage** - compiler may reuse `__TMP0-5` between statements

Debugging Assembly Output

To see the exact mangled names, compile with the `-S` flag and inspect the `.s` output:

```

python compiler.py myprogram.zap -o myprogram.s
cat myprogram.s | grep "^_"      # See all source symbols

```

```
cat myprogram.s | grep "^__" # See all compiler-generated symbols
```

Calling Procedures from Assembly

All procedures and functions are prefixed with `_` in assembly. See [Assembly Label Naming Convention](#) for complete details. Note: parameter passing is register-optimized and the register assignment (A, X, Y) can change between compilations. When calling ZAP procedures **from inside an `asm` block**, check the generated assembly for the actual calling convention. Avoid calling procedures with more than 3 bytes of total parameters from inline assembly — the compiler has no way to verify correctness of hand-written calls. Accessing local variables by name (e.g. `_PROCNAME$VAR`) is always safe.

```
proc setup()
    ; Setup code
end

proc main()
    asm
        JSR _SETUP    ; Call setup procedure (note the _ prefix)
    end
end
```

Assembler-only directives

Assembler directives like `.segment` and `.incbin` are only recognized inside `asm ... end` blocks and are not supported as top-level ZAP directives. Example:

```
proc load_font()
    asm
        .segment "FONT"
        .incbin "font.dat"
        .incbin "font.dat"
    end
end
```

Note: The compiler automatically restores the CODE segment after each `asm ... end` block, so you do not need to manually add `.segment "CODE"` at the end.

For compile-time diagnostics, ZAP provides the following directives (used outside of `asm` blocks):

```
.error "This is a compile error"    ; Emits an error and stops compilation
.warning "This is a warning"        ; Emits a warning but continues
compilation
.info "Informational message"       ; Emits an info message but continues
compilation
```

Assembly Gotchas

Segment switching is safe — the compiler restores the CODE segment automatically:

```
asm
    .segment "DATA"
    .byte 1, 2, 3
    ; No need to restore CODE segment – the compiler does it for you
end
```

Be careful with labels:

```
asm
    JMP MY_LABEL      ; Unique names only
MY_LABEL:
    RTS
end
```

Don't use END in assembly:

```
asm
    LOOP: INC A
    JMP LOOP
    END              ; ERROR: This ends the ASM block!
end
```

Hardware Access

Atari 8-Bit Registers

The Atari has memory-mapped hardware registers:

```
; GTIA (Graphics) Registers
byte GTIA_M0PL @$D00C #PORT #RD    ; Missile 0 / Player collision (read-only)
byte GTIA_P0PL @$D00D #PORT #RD    ; Player 0 collision (read-only)
byte GTIA_HPOS0 @$D000 #PORT #WR    ; Player 0 horizontal position (write-only)

; ANTIC (Display) Registers
word ANTIC_DLST @$D402 #PORT        ; Display list pointer (read/write)
byte ANTIC_HSCROL @$D404 #PORT #WR  ; Horizontal scroll (write-only)

; POKEY (Sound/I/O) Registers
```

```
byte POKEY_AUDF1 @$D200 #PORT #WR ; Audio frequency 1 (write-only)
byte POKEY_AUDC1 @$D201 #PORT #WR ; Audio control 1 (write-only)
```

The **#PORT** modifier marks a variable as a hardware port. The compiler uses this to:

- Skip optimization of port accesses (each read/write must happen as written)
- **#RD** restricts the port to read-only (compile error if you try to write)
- **#WR** restricts the port to write-only (compile error if you try to read)
- **#PORT** alone (without **#RD**/**#WR**) allows both read and write

Reading Registers

```
proc check_collision()
    byte collision = GTIA_M0PL
    if collision != 0
        ; Collision detected!
    end
end
```

Writing to Registers

```
proc move_player(byte x)
    GTIA_HPOS0 = x ; Set player X position
end

proc set_display_list(word address)
    ANTIC_DLIST = address
end
```

Sound Output

```
proc play_note(byte frequency, byte duration)
    POKEY_AUDF1 = frequency
    POKEY_AUDC1 = $A8 ; Enable audio

    ; Wait
    byte i
    for i = 0 to duration
        ; Busy-wait
    end

    POKEY_AUDC1 = 0 ; Stop audio
end
```

Interrupt-like Patterns

Monitor register changes:

```
byte last_joystick = 0

proc poll_joystick()
    byte joystick = POKEY_STICK0    ; Read joystick

    if joystick != last_joystick
        ; Joystick changed
        handle_input(joystick)
    end

    last_joystick = joystick
end
```

Game Loop with Hardware

```
proc game_loop()
    while game_running
        ; Update physics
        update_game()

        ; Check collisions
        check_collisions()

        ; Poll input
        poll_joystick()

        ; Render
        draw_frame()
    end
end
```

Optimization Techniques

Compiler Optimizations

The compiler applies several passes:

```
# Basic compilation
python compiler.py program.zap

# With all peephole optimizations
python compiler.py --peepholes program.zap
```

```
# For 6502 (older CPU, may be slower)
python compiler.py -6502 program.zap
```

Constant Folding

Compiler evaluates constants at compile-time:

```
const byte SIZE = 100
byte data[SIZE + 50]    ; Array size = 150 (compile-time)

proc init()
    byte x = 2 + 3 * 4    ; x = 14 (computed at compile-time)
end
```

Compiler generates:

```
data:    .res 150        ; Not calculated at runtime
x:       .byte 14        ; Not calculated at runtime
```

Dead Code Elimination

Unreachable code is removed:

```
proc dce_example()
    return
    byte unused = 42    ; Never reaches here - removed!

    if 0                ; Condition always false
        unused = 1      ; Removed
    end
end
```

Loop Unrolling for Small Loops

Manual optimization:

```
; Original (5 iterations)
byte i
for i = 0 to 4
    process(i)
end

; Unrolled (faster, more code)
process(0)
process(1)
```

```
process(2)
process(3)
process(4)
```

Lookup Tables vs Calculation

```
; Slow: calculated each time
func byte sin_approx(byte angle)
    return (angle * 127) / 128
end

; Fast: lookup table (pre-computed)
byte sine_table[] = {0, 13, 26, 38, 50, ...}

proc sin_table_lookup()
    byte angle = 10
    byte value = sine_table[angle]
end
```

Cache Data in Registers

```
proc process_loop()
    byte index = 0
    byte temp = data[index]

    for index = 0 to 255
        process_value(temp)
        temp = data[index + 1] ; Pre-load next
    next index
end
```

Struct Array Index Multiply Optimization

When indexing into an array of structs, the compiler must scale the index by the element size. ZAP! uses the most efficient code sequence available for the element size known at compile time.

Power-of-2 element sizes (4, 8, 16, 32 bytes) — pure shifts:

```
struct Sprite ; 16 bytes
    word x
    word y
    byte tile
    byte flags
    byte f0[10]
end

Sprite sprites[32] @$5000
```

```
proc update(byte i)
    sprites[i].tile = 5      ; index*16 via 4 shifts – 14 instructions total
end
```

Generated code for `i * 16`:

```
STA TMP3      ; save index
LDA #$00
ASL TMP3      ; ┌
ROL A         ; │ × 4 (log2 16 = 4 shifts)
ASL TMP3      ; │
ROL A         ; │
ASL TMP3      ; │
ROL A         ; │
ASL TMP3      ; │
ROL A         ; └
TAX           ; X = high byte of offset
LDA TMP3      ; A = low byte of offset
```

Non-power-of-2 element sizes (3, 5, 6, 10, 12 bytes) — shift-add decomposition:

For struct sizes whose binary representation has ≤ 3 set bits, the compiler decomposes the multiply into shifts and adds. For example, size 12 = 8 + 4:

```
struct Entry      ; 12 bytes
    byte key[8]
    word value
    word next
end
```

`index * 12` emits: shift to $\times 4$, add; shift again to $\times 8$, add. About 20 instructions vs 60 for a repeated-add loop.

Compile-time constant indexes — when the array index is a compile-time constant expression (e.g., `SCREEN_Y_SIZE - 1` where `SCREEN_Y_SIZE` is a `const`), the compiler evaluates the entire byte offset at compile time and emits a direct two-instruction load. This works for simple arrays, struct arrays, and multi-dimensional arrays:

```
const byte ROWS = 24
word vlstart[ROWS] @500000

proc update()
    word ptr = vlstart[ROWS - 1] ; offset = (ROWS-1)*2 = 46 = $2E –
    compile-time
```

```
; Emits:  LDA #$2E / LDX #$00  – no runtime multiply needed
end
```

Multiply by Small Constant (Shift-Add)

The compiler uses shift-add decomposition for multiplying a **byte** variable by a small non-power-of-2 constant with at most 3 set bits. This replaces the general **JSR MUL8** routine call with inline shifts and adds — typically 3× faster:

Expression	Decomposition	Code
a * 3	2 + 1	shift-add, ~6 instr
a * 5	4 + 1	shift-add, ~8 instr
a * 6	4 + 2	shift-add, ~10 instr
a * 10	8 + 2	shift-add, ~10 instr
a * 12	8 + 4	shift-add, ~10 instr

Power-of-2 constants (***2**, ***4**, ***8**, ...) use pure shifts (even fewer instructions). Constants with more than 3 set bits fall back to **JSR MUL8**.

Byte vs Word

```
; Use byte when possible (faster, smaller)
byte small_value = 100

; Use word only if necessary
word large_value = 40000
```

Minimize Pointer Dereference

```
; Slow: multiple dereferences
byte ^ptr = @data
byte v1 = ptr^
byte v2 = ptr^
byte v3 = ptr^

; Faster: cache value
byte ^ptr = @data
byte value = ptr^
byte v1 = value
byte v2 = value
byte v3 = value
```

Profile Your Code

Check generated assembly for hotspots:

```
python compiler.py program.zap -o program.s
# Review program.s for performance-critical sections
```

Module System Deep Dive

Module Structure

library.zaplib - Reusable module:

```
.module "math"

func byte abs(byte x)
    if x < 0
        return -x
    end
    return x
end
```

program.zap - Main program:

```
.include "library.zaplib"

proc main()
    byte result = abs(-5)
end
```

Multi-Level Includes

```
main.zap
├─ lib_graphics.zaplib
│   └─ lib_colors.zaplib
├─ lib_math.zaplib
└─ lib_physics.zaplib
    └─ lib_math.zaplib (already loaded)
```

Symbol Visibility

Included modules' symbols become available:

```
; vector.zaplib
func word cross_product(byte ax, byte ay, byte bx, byte by)
    return (ax * by) - (ay * bx)
end

; main.zap
.include "vector.zaplib"

proc main()
    word result = cross_product(1, 2, 3, 4)
end
```

Include Organization

Best practices:

```
project/
├─ main.zap
├─ lib/
│   ├── graphics.zaplib
│   ├── sound.zaplib
│   ├── physics.zaplib
│   └─ util.zaplib
└─ game/
    ├── player.zaplib
    ├── enemies.zaplib
    └─ items.zaplib
```

Avoiding Circular Dependencies

Bad:

```
; a.zaplib
.include "b.zaplib"

; b.zaplib
.include "a.zaplib"    ; Circular!
```

Good:

```
; a.zaplib
func byte func_a() ... end

; b.zaplib
func byte func_b() ... end
```

```
; main.zap
#include "a.zaplib"
#include "b.zaplib"
```

Standard Library Modules

ZAP! ships with a ready-to-use standard library in `work/lib/`. Include the modules you need:

```
.include "string.zap"    ; memcpy, memset, strlen, strncmp, ...
#include "stdio.zap"     ; puts, putchar, getchar, fopen, fwrite, ...
```

The `stdio` module uses conditional compilation to select the right platform backend:

```
-D ATARI → lib/atari/atari_stdio.zap  (screen + CIO file I/O)
-D SBC   → lib/sbc/sbc_stdio.zap     (not yet implemented)
```

Standard library dependency tree:

```
errno ← types ← string
                ↑
errno ← types ← stdio ← [ATARI] atari_stdio
```

For hardware register access, include the chip definition files directly:

```
.include "atari/atari_gtia.zap"    ; GTIA: sprites, color, collision
($D000)
#include "atari/atari_pokey.zap"    ; POKEY: sound, keyboard, paddles
($D200)
#include "atari/PIA.zap"            ; PIA: parallel I/O, joystick dirs
($D300)
```

These files have no `.module` declaration — they are plain include files with no module name.

See [STDLIB.md](#) for the complete API reference.

Performance Profiling

Analyzing Generated Assembly

```
python compiler.py program.zap -o program.s
```

Look for:

- Excessive address calculations
- Repeated dereferences
- Inefficient loop bodies

Code Size Analysis

```
; program.s - Check for:  
; - Size of procedures  
; - Repetitive patterns  
; - Unused code (optimized away)
```

Timing Critical Sections

Count assembly instructions:

```
proc critical_inner_loop()  
    byte i, sum = 0  
  
    ; This loop is critical - minimize instructions  
    for i = 0 to 255  
        sum = sum + data[i]  
    end  
end
```

Memory Usage

Track variable allocation:

```
Zero-page usage:  
- Pointers: ~10 bytes  
- Variables: ~20 bytes  
- Temporaries: ~5 bytes  
Total: ~35 bytes (plenty remaining)  
  
High RAM:  
- Arrays: ~1KB
```

Advanced Patterns

State Machine

```
const byte STATE_INIT = 0
const byte STATE_PLAY = 1
const byte STATE_PAUSE = 2
const byte STATE_OVER = 3

byte game_state = STATE_INIT

proc update_state()
  switch game_state
    case STATE_INIT
      initialize_game()
      game_state = STATE_PLAY
      break

    case STATE_PLAY
      update_game()
      check_pause()
      check_game_over()
      break

    case STATE_PAUSE
      check_unpause()
      break

    case STATE_OVER
      show_game_over()
      break
  end
end
```

Event Handling

```
; Event flags
byte event_collision = 0
byte event_powerup = 0
byte event_death = 0

proc handle_events()
  if event_collision
    on_collision()
    event_collision = 0
  end

  if event_powerup
    on_powerup()
    event_powerup = 0
  end

  if event_death
    on_death()
  end
end
```

```
        event_death = 0
    end
end
```

Object Pooling

```
const byte MAX_SPRITES = 10

byte sprite_x[MAX_SPRITES]
byte sprite_y[MAX_SPRITES]
byte sprite_active[MAX_SPRITES]
byte sprite_count = 0

proc spawn_sprite(byte x, byte y)
    if sprite_count < MAX_SPRITES
        sprite_x[sprite_count] = x
        sprite_y[sprite_count] = y
        sprite_active[sprite_count] = 1
        sprite_count = sprite_count + 1
    end
end

proc update_sprites()
    byte i
    for i = 0 to sprite_count
        if sprite_active[i]
            sprite_y[i] = sprite_y[i] + 1

            if sprite_y[i] > 191
                sprite_active[i] = 0
            end
        end
    end
end
```

Message Passing

```
const byte MAX_MESSAGES = 5
byte message_queue[MAX_MESSAGES]
byte message_count = 0

proc send_message(byte msg)
    if message_count < MAX_MESSAGES
        message_queue[message_count] = msg
        message_count = message_count + 1
    end
end

proc process_messages()
```

```

    byte i
    for i = 0 to message_count
        process_message(message_queue[i])
    end
    message_count = 0
end

```

Frame Timing

```

byte frame_counter = 0

proc game_update()
    frame_counter = frame_counter + 1

    ; Every 30 frames
    if frame_counter == 30
        do_expensive_operation()
        frame_counter = 0
    end

    ; Do fast updates every frame
    do_fast_operation()
end

```

Debugging Assembly Output

Reading Generated Assembly

```

; C:\path\to\program.zap 4: proc main()
; -- Procedure MAIN --
_MAIN:
    ; Actual assembly instructions
    JSR _FUNCTION    ; Call function (note _ prefix for source symbols)
    RTS              ; Return

```

Following Execution

```

LDA #10            ; Load 10 into accumulator
STA _VAR           ; Store into variable (note _ prefix)
; At this point, _VAR = 10

```

Analyzing Procedure Calls

```
JSR _SETUP      ; Jump to subroutine (saves return address)
                ; ... _SETUP executes ...
                ; RTS (return to here)

JMP _LOOP       ; Jump (no return)
```

Checking Variable Addresses

```
.segment "ZEROPAGE"
_MY_VAR: .res 1      ; _MY_VAR at some ZP address
_MY_PTR: .res 2      ; _MY_PTR at ZP address
```

Compiler Internals

Compilation Pipeline

1. **Tokenization** - Convert source to tokens
2. **Parsing** - Build AST
3. **Semantic Analysis** - Type checking, symbol resolution
4. **Optimization** - Constant folding, DCE, etc.
5. **Code Generation** - Emit assembly
6. **Assembly Optimization** - Peephole, jump threading

How Variables Are Named

Global byte x	→ <code>_X</code>
Local byte x in TEST	→ <code>_TEST_X</code>
Pointer ptr	→ <code>_PTR</code> (2 bytes)
Procedure P1	→ <code>P1</code> (label)
Parameter in P1	→ <code>_P1_PARAM</code>

Math Runtime Registers (6502)

The compiler uses a dedicated zero-page math area when lowering `*`, `/`, and `%`:

```
.segment "ZEROPAGE"
MATH_STACK: .res 8      ; 4x 16-bit slots for expression evaluation
MATH0:      .res 4      ; 32-bit accumulator (result)
MATH1:      .res 2      ; 16-bit operand
```

- `MATH0` holds the final result (low word in `MATH0/MATH0+1`).
- `MATH1` holds the second operand for math routines.

- **MATH_STACK** is used to spill intermediate operands in complex expressions.
- Runtime math routines use **TMP0** . **TMP4** as scratch and will reserve them when emitted.

Accumulator-Mode Arithmetic (ADD16/SUB16)

For better efficiency with chained arithmetic, the compiler uses **ADD16** and **SUB16** runtime routines:

```
; ADD16 - Accumulator-style 16-bit addition
; Input:  MATH0 (left operand), MATH1 (right operand)
; Output: MATH0 (sum)
ADD16:
    LDA MATH0
    CLC
    ADC MATH1
    STA MATH0
    LDA MATH0+1
    ADC MATH1+1
    STA MATH0+1
    RTS

; SUB16 - Accumulator-style 16-bit subtraction
; Input:  MATH0 (left operand), MATH1 (right operand)
; Output: MATH0 (difference)
SUB16:
    LDA MATH0
    SEC
    SBC MATH1
    STA MATH0
    LDA MATH0+1
    SBC MATH1+1
    STA MATH0+1
    RTS
```

When evaluating expressions like **a + b * (c + 1) - 10**:

1. **b * (c + 1)** is evaluated and stored in **MATH0**
2. **ADD16** is called to compute **MATH0 += a**
3. Immediate subtraction handles the final **-10**

This avoids repeated temp spilling compared to fully inline arithmetic, reducing code size and improving instruction cache locality for complex expressions.

Note on SUB16 with immediate constants: The compiler uses SUB16 (subroutine) instead of inline SBC instructions for 16-bit immediate subtractions of constants. This unifies the subtraction ABI and allows for future optimization opportunities where intermediate results are cached in MATH0.

Future Optimization: MATH0 Result Caching

The accumulator-based architecture supports a future optimization: **keeping results in MATH0 across expression chains** to eliminate unnecessary temp shuttling. Currently, each operation loads its result to A/X

for downstream use, then the next operation in a chain loads from A/X back to MATH0 for its computation.

Proposed enhancement:

- Add `result_location` parameter to `gen_expr()` ("A/X" or "MATH0")
- For nested expressions, keep intermediate results in MATH0
- Only load to A/X at expression boundaries (assignments, function calls, external returns)
- Would save ~10+ bytes of ZP temp pressure and improve code efficiency

Implementation notes:

- Infrastructure partially in place (`result_location` parameter added)
- Requires propagating `result_location` through all recursive `gen_expr()` calls
- Needs special handling for each expression type (literals, identifiers, math ops, etc.)
- May require two-pass or lookahead logic to determine when result will be immediately re-used in another math operation

Function Return Type Validation

`FuncAnalyzer` in `sema_func.py` validates every `return` statement against the function's declared return type. The rules are:

Declared type	Return expression	Result
Scalar (BYTE/WORD/LONG)	Any scalar or pointer	Allowed (widening/narrowing)
Pointer (e.g. <code>byte^</code>)	Scalar WORD or pointer	Allowed (pointers are WORD-compatible)
Struct <code>S</code>	Same struct <code>S</code>	Allowed
Struct <code>S</code>	Different struct <code>T</code>	Error
Struct <code>S</code>	Scalar	Error
Scalar	Struct	Error
Any <code>func</code>	Missing expression	Error

Pointers are treated as WORD for type-compatibility purposes, so `func word f()` can `return ptr` and `func byte^ g()` can `return some_word`. Struct returns require exact name match (no structural equivalence).

How Arrays Are Accessed

```
byte arr[5]
byte index = 2
byte value = arr[index]
```

Compiles to:

```

; Calculate: base + index
LDA #<ARR          ; Low byte of array address
CLC
ADC _INDEX          ; Add index
STA TMP0           ; Store in temp
; Load value
LDY #0
LDA (TMP0),Y        ; Indirect addressing

```

Binary Number Literals

```

byte b1 = %10101010    ; Binary
byte b2 = 0b1010        ; 0b prefix
byte h1 = $FF           ; Hex (dollar sign)
byte h2 = 0xFF          ; 0x prefix (decimal also works)
byte d1 = 255           ; Decimal

```

Const Storage and Immutability

The `const` modifier affects how data is stored and whether it can be modified:

Const Scalars — Assembler Equates

Const scalar variables occupy **no memory at all**. They are emitted as assembler symbol equates:

```

const byte MAX = 100
const word SCREEN = $4000

```

Produces:

```

_MAX = $64
_SCREEN = $4000

```

Every use of `MAX` in code is substituted with the literal value at compile time (via `constsubst.py`). There is nothing to modify — no RAM, no ROM address.

Const Arrays and Structs — CODE Segment

Const arrays and const structs are stored as read-only data in the **CODE segment**:

```
const byte table[] = {10, 20, 30, 40, 50}
const Point origin = {0, 0}
```

Produces:

```
; In CODE segment (alongside executable code)
__ARRAY_DATA_1:
    .byte $0A, $14, $1E, $28, $32
__ARRAY_DATA_2:
    .byte $00, $00
```

Unlike non-const arrays (which are copied from ROM to BSS at startup and then read/written in RAM), const arrays are **accessed directly from their CODE segment location**. No RAM copy exists.

Compile-Time Write Protection

The compiler rejects all direct modifications to const data:

```
const byte MAX = 10
const byte table[] = {1, 2, 3}
const Point origin = {0, 0}

MAX = 20           ; ERROR: Cannot assign to const variable 'MAX'
table[0] = 99       ; ERROR: Cannot assign to element of const array
'TABLE'
origin.x = 5        ; ERROR: Cannot assign to field of const struct
'ORIGIN'
```

The compiler also detects writes through pointers when the address-of target is visibly const:

```
(@table + 1)^ = 99 ; ERROR: Cannot write through pointer to const 'TABLE'
```

Runtime Implications: RAM vs ROM

On the 65(c)02 8-bit, a loaded program resides in **RAM**. This means the CODE segment is technically writable at the hardware level. If a pointer to a const array is stored in a word variable (bypassing the compile-time check), writing through it will silently modify the data in RAM:

```
const byte table[] = {1, 2, 3}
word addr = @table    ; Store address in plain word (no const tracking)
byte ^ptr
```

```
ptr = addr          ; Compiler loses track of const origin
ptr^ = 99           ; NO compile error – modifies table[0] in RAM
```

This works because the program runs from RAM. However, if the program is placed in **ROM** (e.g., an Atari cartridge image), writing to const data will have no effect or cause undefined behavior, since ROM is not writable by the CPU.

Best practice: Treat **const** as a logical contract. The compiler enforces it for all direct and most indirect accesses. Do not rely on the ability to bypass it through pointer laundering — future compiler versions may enforce it more strictly.

Best Practices

1. **Profile before optimizing** - Measure first
 2. **Use appropriate types** - byte when possible
 3. **Precompute when possible** - Lookup tables
 4. **Organize with modules** - Keep code maintainable
 5. **Document hardware access** - Comments for registers
 6. **Test incrementally** - Small, verifiable changes
 7. **Check generated assembly** - Understand output
 8. **Use fixed addresses carefully** - Avoid collisions
 9. **Monitor zero-page usage** - Most precious resource
 10. **Leverage compiler optimizations** - Use --peepholes
-

Troubleshooting Advanced Issues

"Zero-page exhausted"

Too many pointers or small variables:

```
; Current:
byte ^ptr1, ^ptr2, ^ptr3 ; 6 bytes
byte x, y, z             ; 3 bytes
; Total: 9 bytes

; Solution: Reduce pointer count
byte ^ptr1               ; Only needed pointer
word address1, address2 ; Use word arrays instead
```

"Circular dependency"

Modules include each other:

```
; Break cycle:
; a.zaplib - defines func_a
```

```
; b.zaplib - defines func_b
; main.zap - includes both
```

Performance unexpectedly poor

```
python compiler.py -o program.s program.zap
# Review program.s for:
# - Repeated dereferences
# - Inefficient loops
# - Unnecessary multiplications
```

Memory Allocation Details

Overview

The ZAP compiler uses sophisticated variable allocation strategies to optimize memory usage on 6502/65C02 systems. This document explains how local variables are allocated, how slot sharing works, and how variables are prioritized for Zero Page allocation.

Memory Segments

Variables are stored in different segments based on their properties and type:

ZEROPAGE Segment (256 bytes, \$0000-\$00FF)

Highest priority: Contains frequently-accessed variables for fast access.

System variables (reserved):

- **MATH_STACK:** 8 bytes - Stack for mathematical operations
- **MATH0:** 4 bytes - Math temporary storage
- **MATH1:** 2 bytes - Math temporary storage
- Temporary registers: **TMP0-TMP5** - 2-12 bytes (allocated as needed)

User variables (allocated after system variables):

- **Shared slots:** Local variables that share storage through aliasing
- **Pointer variables:** ALL pointers must be in zero page (mandatory for fast indexing)
- **Byte variables** (high priority): Short-lived or frequently-used byte locals
- **Word variables** (high priority): Short-lived or frequently-used word locals

BSS Segment (RAM, typically \$0300+)

Lower priority: Overflow variables and large structures.

Contains:

- Word variables that don't fit in zero page

- Byte variables that don't fit in zero page
- Struct variables (all structs go to BSS by default)
- Array variables (all arrays go to BSS, except pointer arrays which go to ZP if they fit)

CODE Segment

Not typically used for data, but may contain:

- Runtime constant data
- String literals (with null terminators)
- Array initialization data (const arrays)

Variable Types and Storage

Fixed-Address Variables

Variables declared with `@address` syntax:

```
word port_a @$FF00
```

These are emitted in the ZEROPAGE segment:

```
byte result = $FF
```

Characteristics:

- Cannot be modified by ZP allocation algorithm
- Used for hardware ports and memory-mapped I/O
- Protected from peephole optimization

Pointers and Pointer Arrays

Requirement: ALL pointers MUST be in zero page.

Rationale:

- 6502 indirect addressing `(addr),Y` requires the address to be in zero page
- Without specific addressing modes, pointers would require 3-4 byte sequences instead of optimal 1-2 byte sequences

Example:

```
proc print(byte *str)
    ; str pointer is in zero page for efficient `(str),Y` addressing
end
```

Scalar Variables (BYTE and WORD)

Local variables that can potentially share storage:

```
proc calculate()  
    byte x = 10  
    byte y = 20  
    word total = 0  
end
```

If **x** and **y** don't overlap in their liveness (x is dead before y is used), they can share the same zero page slot.

Arrays

All non-pointer arrays are allocated in BSS:

```
proc process()  
    byte buffer[256] ; Goes to BSS, too large for ZP  
    byte temp[4]      ; Still goes to BSS (arrays don't fit in ZP)  
end
```

Structs

All struct variables are allocated in BSS:

```
struct point  
    byte x  
    byte y  
end  
  
proc main()  
    point p ; Goes to BSS  
    point *ptr_p = @p ; ptr_p goes to ZP if fits, points to p in BSS  
end
```

Slot Sharing Strategy

What is Slot Sharing?

Slot sharing (or variable aliasing) is when multiple local variables use the same storage location because their liveness ranges don't overlap.

Liveness Analysis

The compiler performs **liveness analysis** on each procedure to determine:

- **Live-in:** Variables that are live at procedure entry
- **Live-out:** Variables that are live at procedure exit
- **Live-gen:** Variables used in this statement
- **Live-kill:** Variables that are dead after this statement

Example of Slot Sharing

```
proc example()
  word x = 100
  word y = 200
  word z = x + y
end
```

All three variables (**x**, **y**, **z**) might share a single 2-byte slot because:

1. **x** is live during **x + y** computation
2. **y** is live during **x + y** computation
3. After assignment to **z**, both **x** and **y** are dead
4. If **z** is not used later, it's also dead

This results in a single `__LVSL0T_1` being used for all three.

Graph Coloring Algorithm

The compiler uses **greedy graph coloring** to assign slots:

```
Input: Interference graph with nodes (variables) and edges (conflicts)
1. Sort nodes by degree (number of conflicts) - descending
2. For each node:
  a. Collect colors used by neighbors
  b. Assign the lowest available color
3. Group by color
4. For each color group with > 1 variable:
  a. Create a shared slot (`__LVSL0T_n`)
  b. Assign it to all variables in that group
```

Shared Slot Naming

Shared slots are named sequentially: `__LVSL0T_1`, `__LVSL0T_2`, etc.

In assembly output:

```
; Shared slots (for aliased locals)
__LVSL0T_1:      .res 2      ; 2 bytes for word-type variables
__LVSL0T_2:      .res 4      ; 4 bytes for array-type variables
```

In procedure code, individual locals are aliased:

```
MAIN:
_MAIN_X = __LVSL0T_1
_MAIN_Y = __LVSL0T_1
_MAIN_Z = __LVSL0T_1
```

This means:

- `_MAIN_X`, `_MAIN_Y`, `_MAIN_Z` all reference the same location
- Assembler replaces all references with `__LVSL0T_1`
- No separate storage is allocated for each variable

Zero Page Prioritization

Priority Levels

Variables are prioritized for zero page allocation using a **frequency score** (`zp_priority`):

Score = loop_depth_weight × access_count

Where `loop_depth_weight` is exponential:

- **Outside loops:** weight = 1
- **Loop depth 1:** weight = 10
- **Loop depth 2:** weight = 100
- **Loop depth 3+:** weight = 1000+

Example Priority Calculation

```
proc loop_demo()
    byte counter = 0          ; zp_priority = 10 (accessed 1x in loop
depth 1)
    byte total = 0           ; zp_priority = 30 (accessed 3x in loop
depth 1)

    while counter < 10
        total = total + counter
        counter = counter + 1
    end
end
```

Priorities:

- `counter`: Loaded and stored twice per iteration (depth 1) → $2 \times 10 = 20$
- `total`: Loaded twice and stored once per iteration (depth 1) → $3 \times 10 = 30$

If space permits, `total` gets priority because it has higher access frequency.

Allocation Order

1. **Fixed-address variables** - Cannot be moved
2. **All pointers** - Mandatory in zero page
3. **High-frequency scalars** - Sorted by priority, descending
4. **Regular scalars** - Fill remaining space
5. **Overflow** - Spill to BSS

Allocation Algorithm

Phase 1: Liveness Analysis

For each procedure:

1. Compute control flow graph (CFG)
2. For each block in reverse order:
 - a. Live-out = union of Live-in of successors
 - b. Live-gen = variables used before being defined
 - c. Live-kill = variables defined in this block
 - d. Live-in = Live-gen $\cup$ (Live-out - Live-kill)
3. Mark variables as live/dead at each instruction

Phase 2: Interference Graph

Build a graph where:

- **Nodes** = local variables (per procedure)
- **Edges** = conflict (two variables are live simultaneously)

```
if (x and y are both alive at any point)
    add edge(x, y)
```

Also add **call-live-across** constraints:

- Variables live across procedure calls interfere with callee locals

Phase 3: Greedy Coloring

```
# Sort by degree (conflicts) descending
variables = sorted(vars, key=lambda v: -len(conflicts[v]))

colors = {}
for var in variables:
    # Find colors used by neighbors
    neighbor_colors = {colors[n] for n in neighbors(var) if n in colors}
```

```
# Assign lowest available color
color = 0
while color in neighbor_colors:
    color += 1
colors[var] = color
```

Phase 4: Slot Assignment

```
# Group variables by color
color_groups = {}
for var, color in colors.items():
    color_groups.setdefault(color, []).append(var)

# For each color group with multiple variables
slot_counter = 0
for color, group in color_groups.items():
    if len(group) > 1:
        slot_counter += 1
        slot_label = f"__LVSL0T_{slot_counter}"
        for var in group:
            var.shared_slot = slot_label
```

Safety Features

Uninitialized Variable Detection

The ZAP! compiler performs **definite-assignment analysis** to detect when local variables are read before being initialized. This catches a common class of bugs at compile time, preventing runtime errors and undefined behavior. Please note that this detection does NOT catch all possible uninitialized variable bugs, but it does catch the most common ones.

What is Detected

The compiler tracks the initialization state of all local variables (including procedure and function parameters) and reports an error if a variable is read before it has been assigned a value.

Reads that are checked include:

- Variable use in expressions, returns, and conditions
- Index expressions (e.g., `arr[i]`, where `i` must be initialized)
- Pointer dereference expressions (the pointer variable must be initialized)
- Any arithmetic or logical use on the right-hand side

Scope of analysis:

- This analysis applies to **local variables** in procedures and functions
- Parameters are treated as initialized at entry

- Globals are not tracked with definite-assignment; they follow normal declaration and initialization rules

Example

```
proc example()  
  byte x  
  byte y = x + 10    ; ERROR: Use of uninitialized variable 'X'  
end
```

This code will fail to compile with the error message:

```
example.zap:3:14: error: Use of uninitialized variable 'X'
```

Initialization Methods

The compiler recognizes several ways a variable can be initialized:

1. Explicit Initializer

Variables with explicit initialization expressions are considered initialized:

```
proc test()  
  byte x = 42          ; x is initialized  
  byte y = x + 10      ; OK: x is initialized, y becomes initialized  
end
```

2. Assignment Statement

Variables become initialized after assignment:

```
proc test()  
  byte x  
  x = 42                ; x becomes initialized here  
  byte y = x + 10      ; OK: x is initialized  
end
```

3. FOR Loop Variables

The loop variable in a FOR loop is considered initialized before the loop body:

```
proc test()  
  byte i  
  for i = 0 to 10      ; i is initialized by the FOR loop  
    result = i        ; OK: i is initialized  
  end  
end
```

Control Flow Handling

The compiler tracks initialization state through control flow constructs:

IF Statements

Variables initialized in one branch are not considered initialized after the IF unless both branches initialize them:

```
proc test(byte flag)  
  byte x  
  
  if flag  
    x = 10  
  end  
  
  result = x          ; ERROR: x not initialized in else branch  
end
```

To fix this, initialize in both branches:

```
proc test(byte flag)  
  byte x  
  
  if flag  
    x = 10  
  else  
    x = 20  
  end  
  
  result = x          ; OK: x initialized in all paths  
end
```

WHILE Loops

The loop body may execute zero times, so the compiler assumes variables initialized only inside the loop are not initialized after:

```
proc test()
  byte x
  byte i = 0

  while i < 10
    x = i          ; x initialized in loop
    i = i + 1
  end

  result = x       ; ERROR: x might not be initialized (loop might not
run)
end
```

REPEAT-UNTIL Loops

The loop body always executes at least once, so variables initialized in the body are considered initialized after:

```
proc test()
  byte x

  repeat
    x = 42
  until x > 100

  result = x       ; OK: x is initialized (loop runs at least once)
end
```

SWITCH Statements

Variables must be initialized in all cases (including DEFAULT) to be considered initialized after the SWITCH:

```
proc test(byte n)
  byte x

  switch n
    case 0
      x = 10
      break

    case 1
      x = 20
      break

    default
      x = 30
      break
```

```

    end

    result = x          ; OK: x initialized in all cases
end

```

Nested Control Flow

Initialization tracking works across nested conditionals and loops. A variable is considered initialized after a nested block only if **all possible paths** within that block assign it.

Special Cases

Address-of Operator

Taking the address of a variable does **not** initialize it. The address-of operator (`&`) is not considered a read operation:

```

proc set_ptr(byte ^ptr)
    ptr^ = 99
end

proc test()
    byte x
    set_ptr(&x)          ; OK: taking address doesn't require initialization
    result = x           ; ERROR: x not initialized (taking address doesn't
count)
end

```

However, if you initialize the variable before passing its address, it's valid:

```

proc test()
    byte x = 0           ; Initialize first
    set_ptr(&x)          ; OK
    result = x           ; OK: x was initialized
end

```

Subscript and Field Access

Array subscripts and struct field accesses are considered reads only of the base variable (for the address), not the contents. The index expression is fully checked:

```

proc test()
    byte arr[10]
    byte i

```

```
    arr[i] = 42          ; ERROR: i is uninitialized
end
```

The array `arr` itself doesn't trigger an error because we're computing its address, not reading its value. But the index `i` must be initialized.

Field access on structs is treated similarly: the base struct variable is considered initialized (see below), while any index or expression inside the access is fully checked.

Const Variables

Const variables are always considered initialized since their values are known at compile time:

```
proc test()
    const byte x = 42
    result = x          ; OK: const variables are always initialized
end
```

Fixed-Address Variables

Variables declared with fixed addresses (hardware ports) are considered always initialized:

```
byte PORT_A @$D000

proc test()
    result = PORT_A     ; OK: fixed-address variables are always initialized
end
```

Struct Variables

Struct variables are always considered initialized because:

1. They can be partially initialized through field-by-field assignments
2. They can be initialized through pointers (alias analysis is complex)
3. Tracking individual field initialization would require sophisticated analysis

```
struct Point
    byte x
    byte y
end

proc test()
    Point p
    Point ^pp = @p
    pp^.x = 3
    pp^.y = 4
```

```
    result = p.x      ; OK: structs are considered initialized
end
```

Parameters

Procedure and function parameters are always considered initialized (they receive values from the caller):

```
proc test(byte x)
    result = x      ; OK: parameters are always initialized
end
```

What Is Not Detected

The current analysis is intentionally simple and does **not** attempt to prove every safe case. It errs on the side of caution and may report false positives in these scenarios:

- **Inter-procedural initialization:** If a procedure initializes a variable via pointer, the caller will still see it as uninitialized unless it was already initialized.
- **Pointer aliasing:** The compiler does not track that `ptr^ = 42` initializes the pointed-to variable.
- **Partial struct initialization:** The analysis does not track per-field initialization.
- **Conditional pointer writes:** Pointer-based assignments are not recognized as definite initialization.
- **Globals:** Local definite-assignment tracking does not extend to globals.

Comparison with Other Languages

C/C++

Most C/C++ compilers warn about uninitialized variables but don't enforce it:

```
// C code - compiles with warning
void example() {
    int x;
    int y = x + 10;    // Warning: 'x' is used uninitialized
}
```

ZAP! makes this a **compile-time error**, preventing the code from compiling.

Rust

Rust enforces definite-initialization strictly:

```
// Rust code - does not compile
fn example() {
    let x: i32;
```

```
    let y = x + 10;    // Error: borrow of possibly-uninitialized variable
}
```

ZAP!'s approach is similar to Rust's, catching errors at compile time.

Java/C#

These languages initialize all variables to zero/null by default, avoiding the problem but with a runtime cost. ZAP! requires explicit initialization for better performance and cleaner semantics.

Summary

- ZAP! detects uninitialized variable reads at **compile time**
- Variables must be initialized through **explicit initializers, assignments, or FOR loops**
- Control flow is tracked: initialization must occur in **all paths**
- Special cases: **const, fixed-address, structs, and parameters** are always considered initialized
- **Address-of** operator doesn't require initialization
- Errors are reported with precise **file, line, and column** information

This feature prevents a common source of bugs and makes ZAP! programs more reliable.

Debugger Symbol Support

The ZAP! compiler now generates debug information that allows you to debug your compiled programs using emulator debuggers like VICE (Commodore emulator) and Oricutron (Oric emulator) with meaningful symbol names instead of raw hex addresses.

What Is Debugger Symbol Support?

Debugger symbol support enables your debugging tools to:

- **Reference code by procedure/function names** instead of hex addresses
- **Set breakpoints by symbol name** (e.g., `break _main` instead of `break $4006`)
- **Inspect variables by name** in memory
- **View disassembly with label names** for better readability
- **Understand program structure** through symbol information

How It Works

Generation Pipeline

```
ZAP Source Code
  ↓
Compiler generates assembly with .DEBUGINFO directive
  ↓
ca65 assembler (-g flag) embeds debug info in object file
  ↓
ld65 linker generates .lbl or .sym label file
```

```
↓  
VICE/Oricutron loads label file  
↓  
Debugging with symbol names enabled!
```

Automatic Features

1. **.DEBUGINFO + Directive** - Automatically emitted in generated assembly
 - Tells ca65 assembler to include all symbols (not just exports)
 - Includes local labels, procedures, functions, and variables
2. **Assembly with Debug Info** - Build process uses **-g** flag
 - **ca65 -g** embeds symbol information in object files
 - Increases object file size by ~30% (acceptable trade-off)
3. **Label File Generation** - Linker generates symbol mapping
 - VICE: Creates **.lbl** file with symbol → address mappings
 - Oricutron: Creates **.sym** file with symbol → address mappings
 - Automatically generated during **make** or **make.bat**

Usage

Building with Debug Symbols

Using Make (Linux/macOS):

```
make atari          # Generates p1.com and p1.lbl
```

Using make.bat (Windows):

```
make.bat atari      # Generates p1.com and p1.lbl
```

Both will generate:

- **out/p1.com** - Your compiled Atari executable
- **out/p1.lbl** - Label file with debug symbols (VICE format)

Debugging with VICE

1. Start VICE with your program:

```
x64sc -moncommands out/p1.lbl out/p1.prg
```

2. Or load labels in the VICE monitor:

```
ll "out/p1.lbl"
```

3. Use symbol names in the monitor:

```
# Disassemble from a procedure
d ._main

# Set breakpoint at procedure
break ._main

# Display variable contents
print _my_variable

# Jump to address by name
jump ._my_procedure
```

Debugging with Oricutron

1. Start Oricutron with your program:

```
oricutron -symfile out/p1.sym out/p1.com
```

2. Or load symbols in the Oricutron monitor:

```
sl out/p1.sym
```

3. Use symbol names:

```
# Disassemble from a procedure
d ._main

# Set breakpoint
break ._main
```

Symbol Naming Conventions

Symbols generated from your ZAP code follow cc65 conventions:

- **Procedures:** `_proc_name` (underscore prefix for public symbols)
- **Functions:** `_func_name`

- **Global variables:** `_var_name`
- **Local variables:** Generated with scope qualifiers (visible to debugger)
- **Labels:** Internal labels like `L1`, `L2`, etc.

troubleshooting

Label file not generated

Problem: No `.lbl` file appears after build

Solutions:

1. Check that `make atari` or `make.bat atari` completes without errors
2. Verify `out/` directory exists and is writable
3. Ensure `ld65` has `-Ln` option (should be automatic)

Symbols not recognized in VICE

Problem: VICE says "Unknown command" when using symbol names

Solutions:

1. Ensure label file is loaded: `ll "path/to/p1.lbl"`
2. Check that symbols have `.` prefix in VICE: `d ._main`
3. Verify label file path is correct and file exists